Relay: Department Operating Copilot

Working name: **Relay**. It maps the whole department (the meeting memory graph), carries its operational weight (tasks, reports, agents), and scales to bear more departments later. Alternates are in the cover note. Easy to swap.

A self-improving operating copilot for a department. It runs the daily, weekly, and monthly reporting rhythm, turns meeting notes into assigned tasks, keeps a queryable memory of every meeting, catalogs the agents your team builds, tracks AI tool spend by person and department, pushes notifications into Microsoft Teams, captures every human correction as eval data, and improves its own guidelines from human-approved feedback. It exposes its own architecture, workflows, and guidelines as an in-app reference that everyone reads and only Admin or L3 edits. Access is through company SSO only. It runs in two environments, beta and prod. Observability is in the foundation, and the platform is built to add more departments and new flows without a rewrite.

---

0. Architecture at a glance

Seven layers, top to bottom.

1. **Experience (views).** Tasks, Daily and Weekly and Monthly reports, Second Brain, Agent Farm, Expense Tracker, Codex, Admin dashboard, L3 dashboards, and the Concierge chat box.
2. **Agents.** Scribe, Dispatcher, Cartographer, Concierge, Nudge, Rollup, Ledger, Bursar, Curator, Sentry, Quartermaster, the Meter service, and Briefer (next version).
3. **Orchestration.** LangGraph flows with human-approval interrupts, on a durable scheduler, driven by versioned workflow definitions.
4. **Core services (domain).** Permission and department scope, Tasks, Reporting, Guidelines, Memory graph, Codex registry, Engram capture, and the Notification layer.
5. **Integration and access.** Company SSO for login, Microsoft 365 through Graph for Teams and Outlook, OpenRouter API, Anthropic Team usage, and a secret vault.
6. **Data.** Postgres as the single source of truth with pgvector, the cost ledger and model pricing, the Engram interaction and eval store, and the observability backend.
7. **Cross-cutting.** Observability and cost, the self-evolving loop (Engram to Curator to human gate), department scoping, Codex transparency, two environments with eval-gated promotion, and security.

Main loop: agents draft, humans review and correct, Engram stores the draft versus the final, the Curator reads the week of corrections to propose better guidelines, Admin approves, and

the change is tracked in the Codex. The same corrections become eval sets that gate every promotion from beta to prod.

---

1. Assumptions to confirm

1. Admin owns permission config and approves guideline changes. Admin and L3 may be one login with two grants.
2. Self-evolving means human-gated. Agents propose, humans approve, approvals train the agents.
3. Monthly worklog is approved by the employee's direct manager (an L2 who reports to you).
4. Scope follows the reporting tree (`manager_id`) and department.
5. Department is a first-class boundary from day one.
6. Tool spend comes from OpenRouter, Claude Team accounts, and Outlook billing emails.
7. The Codex makes architecture, workflows, and guidelines read-for-all and edit-for-Admin-or-L3.
8. Login is company SSO only, restricted to the company email domain.
9. Two environments: beta for testing, prod for users.

---

2. Design principles (opinionated)

Agents where there is judgment, deterministic services elsewhere. No cap on agent count.

One source of truth. Tasks, reports, memory, spend, guidelines, workflows, and interactions all live in one database per environment.

One definition per workflow, two consumers. The engine runs the same workflow record the Codex renders, so the reference cannot drift from the app.

Every human correction is data. When a person accepts, edits, or rejects an agent draft, store the input, the draft, the action, and the final. This is the eval set and the training signal. Capturing it is not optional and starts with the first reviewable action.

The system is transparent about itself. Architecture, workflows, and rules are readable by everyone. Only Admin or L3 change them. Every change is versioned and visible.

Beta and prod are isolated, and evals gate promotion. Beta never touches real users, real Teams chats, real mail, or prod spend. A change reaches prod only after its eval set passes.

Permissions, department, and identity are layers. Every request passes a scope check, and every login passes company SSO.

Observability is foundational. Tracing, metrics, and cost are wired in before any agent ships.

Secrets live in a vault. Least privilege, read-only mail.

Human gate on evolution. Agents propose, a human approves, and the decision is stored.

---

3. Roles and permission matrix

Login is company SSO only (section 13). The SSO identity maps to an employee record, which sets role and scope.

Capability	L1	L2	L3	Admin
View own tasks	Yes	Yes	Yes	Yes
View subordinate tasks	No	Direct + indirect	Dept-wide	All
Fill daily / submit weekly / submit monthly	Yes	Yes	Yes	No
Approve and edit weekly report	No	For reportees	For all	No
Approve / reject monthly worklog	No	Immediate reportees only	View only	No
See blocked tasks	Own	Team	Dept-wide	All
Task analytics	No	Team	Dept-wide	Dept-wide
Query Second Brain	Own + subtree	Subtree	Dept-wide	Dept-wide
View / edit own Agent Farm card	Yes	Yes	Yes	Yes
Edit any Agent Farm card	No	No	Yes	Yes
View tool expense (own / team / dept)	Own	Team	Dept-wide	All depts
View Codex (all, with history)	Yes	Yes	Yes	Yes

Edit Codex pages, guidelines, workflows	No	No	Yes	Yes
View eval sets and quality dashboards	No	Team	Yes	Yes
Cost and observability dashboards	No	Team health	Yes	Yes
Nudge employees	No	Team (optional)	Yes (optional)	Yes
Approve / reject guideline proposals	No	No	Optional	Yes
Configure permissions, connectors, departments, environments	No	No	No	Yes

Whenever a person edits an agent draft (for example an L2 editing a weekly summary before approving), the edit is captured in Engram regardless of role.

---

4. Core data model

Single Postgres database per environment. pgvector for search. Secrets in a vault, referenced by id.

Org and access:

- **departments**: id, name, kind (line / service).
- **employees**: id, name, employee_id, department_id, sub_department, role_level, manager_id, email, teams_id, sso_subject (stable id from the IdP).

Work and reporting: **tasks** (with status Backlog, Todo, In Progress, Blocked, Done, Overdue, and source mom/manual/derived/ticket/checkin), **moms**, **action_items**, **daily_reports**, **weekly_reports**, **monthly_worklogs**, **app_feedback**, **nudges**. Each carries department_id.

Codex: **guidelines** (versioned, feeds agents), **reference_docs** (architecture and knowledge), **workflow_defs** (engine runs, Codex renders), **reference_revisions** (visible history), **guideline_proposals**.

Engram (interaction memory and eval):

- **interactions**: id, trace_id, flow, agent, employee_id, department_id, input_ref, draft_output, human_action (accept / edit / reject), final_output, edit_diff, reason, created_at. One row per reviewed agent output.
- **eval_sets**: id, agent, name, version, source_window, size, created_at.

- **eval_examples**: id, eval_set_id, interaction_id, input_context, prediction (agent draft), gold (human final), label (accept / edit / reject), notes.
- **agent_runs** stays as the light operational summary and links to interactions by trace_id.

Memory graph: **graph_nodes**, **graph_edges**.

Agent Farm: **farm_agents**. Agentic Gains: **agent_usage** (farm_agent_id, period, task_category, output_category, units_processed, agent_time, source), **agent_gains** (farm_agent_id, period, baseline_hours, agent_assisted_hours, hours_saved).

Cost and observability: **cost_ledger**, **model_pricing**.

External spend: **tool_accounts**, **tool_spend**.

Notifications: **notification_log**, **channel_config**.

Reserved seams: **tickets** (cross-department service requests), **checkin_briefs** (monthly L2 check-ins, section 15).

Metric mapping in config: metric, activity, task, output categories, and unit_economics.

---

5. Agent roster

L0 suggest, L1 act with approval, L2 auto within guardrails.

Agent	Job	Trigger	Autonomy
Scribe	Extract action items from a MOM	New MOM	L0
Dispatcher	Match items to people, draft tasks	After Scribe	L1
Cartographer	Build and maintain the memory graph	After Scribe	L2
Concierge	Chat: how-to, feedback, Second Brain query	User opens chat	L2
Nudge	Chase missing daily reports via the channel layer	Daily and Admin	L2
Rollup	Derive weekly reports	Weekly	L0 then L1
Ledger	Compile monthly worklogs	Month end	L0

Bursar	Pull tool spend, parse invoices, attribute, flag anomalies	Daily	L2 ingest, L0 anomaly
Curator	Read the week of Engram corrections, propose guideline edits with evidence	Weekly	L0
Sentry	Surface blocked and overdue tasks	Continuous	L2
Quartermaster	Draft Agent Farm cards, flag dead links	On add and weekly	L0
Meter	Log tokens and cost per LLM call	Every call	System service
Briefer (next version)	Compile a monthly check-in brief per L2 (section 15)	Monthly	L0

Every agent that produces a human-reviewed draft writes to Engram on review. The Curator now reads Engram, not just task data, which is what turns the L2 edits of weekly summaries into guideline proposals.

---

6. Flow by flow design

The eight core flows are unchanged in shape. Two clarifications. First, in the weekly flow the approver (the L2) can edit the Rollup draft before approving, and that edit is stored in Engram as draft versus final. Second, the Curator's weekly run reads the Engram corrections across all flows, clusters the patterns, and sends guideline proposals to Admin, whose decisions are tracked in the Codex. Each flow's definition lives in `workflow_defs` and renders in the Codex.

---

7. Second Brain view

Queryable meeting memory plus a self-maintained Obsidian-style graph, built by Cartographer after each MOM. Timeline, force graph, and per-note views. GraphRAG query through Concierge with citations. Postgres plus pgvector first, Neo4j later if needed.

---

8. Agent Farm

A shared catalog of the agents your team builds. Cards with owner, name, description, level, scope, quick steps, deployed link, and hours saved per month, plus health checks. Quartermaster drafts cards on demand. The Farm also collects each agent's usage stats so the Agentic Gains view can compute savings.

How the Farm gets usage stats. Two ways, in order of preference. First, if a team agent routes its model calls through the shared gateway (the same one Meter wraps), Relay sees its runs automatically. Second, for agents that cannot use the gateway, expose a small report endpoint each agent calls after a run: agent id, task category, output category, units processed, and time taken. Either way the data lands in `agent_usage`.

8b. Agentic Gains

The payoff view. For every agent in the Farm, it shows the hours per month it saves the team, with a per-owner and per-department leaderboard and a team total.

How savings are computed. A plain calculation, not a guess. For each task type an agent handled, take the manual baseline time per unit from your unit economics, subtract the agent-assisted time per unit, and multiply by the units the agent did.

```
hours_saved = sum over task types of (baseline_manual_hours_per_unit
minus agent_assisted_hours_per_unit) times units_done
```

Agent-assisted time is the agent's own processing time plus any human review time captured in Engram for that work. This ties three things you already have: the Farm's usage stats, the unit economics baselines, and Engram's review records. The Gains service does the arithmetic on a schedule and writes `agent_gains`. No LLM agent is needed, because this is computation, not judgment.

Honest caveat. The numbers are only as good as two inputs: accurate unit-economics baselines and accurate usage reporting from each agent. Where an agent only partly automates a task, count the saved portion, not the whole task, or the gains will be overstated.

---

9. Codex (system reference and governance)

The in-app single reference, read by everyone, written only by Admin or L3. Holds architecture pages, workflow definitions (rendered from the same records the engine runs), the live guidelines, and supporting knowledge. Every edit writes a visible revision with who, when, why, and whether it came from a human or a Curator proposal. Workflow edits are validate-then-activate, treated like a deploy, and that validation runs in beta against eval sets

before prod (section 14). The Codex is where the self-evolving loop becomes visible to the whole team.

---

10. Engram (interaction memory and eval)

The memory layer that records how people use and correct the app, and turns it into eval data. This is distinct from the Second Brain, which remembers meetings. Engram remembers corrections.

What it captures. Every time a person reviews an agent draft, Engram stores one `interactions` row: the input, the agent draft, the action (accept, edit, or reject), the final version, and the diff. Your example flows straight through: an L2 edits a weekly summary, the edit is stored as draft versus final, with the reason if given.

What it produces.

- **Eval sets.** Interactions become `eval_examples` where the agent draft is the prediction and the human final is the gold answer. Grouped per agent into versioned `eval_sets`. These measure quality over time, catch regressions when you change a prompt or model, and feed DSPy optimization for Scribe, Cartographer, Rollup, and Curator.
- **Guideline proposals.** Weekly, the Curator reads the corrections, finds systematic patterns (for example L2s consistently adding a section to weekly summaries), and proposes a guideline edit with the offending interactions as evidence. Admin approves or rejects, and the change is tracked in the Codex.

Why it matters. The corrections you already make by hand become both the test that protects quality and the signal that improves the rules. Nothing extra to label. The work of reviewing is the labeling.

Guardrails. Engram stores employee inputs, so it is scoped and access-controlled like everything else, retained on a defined window, and never used to rank or penalize individuals. It improves agents and guidelines, not people.

---

11. Observability and internal cost (foundational)

Trace IDs follow each request. OpenTelemetry plus self-hosted Langfuse for traces, prompt versions, and evals. The Meter wrapper writes cost to `cost_ledger` and the trace to Langfuse. Metrics: latency, error rate, retries, token cost per agent and flow. Internal cost dashboard: lifetime spend by flow, agent, user, and month, with tokens. Engram quality metrics (edit rate,

reject rate, eval scores per agent) sit alongside. Route by cost: small model for extraction, stronger for analysis. Store current model prices in `model_pricing`.

---

12. Tool Expense Tracker

Department-wise and person-wise external AI spend, separate from Relay's own model cost. Bursar reconciles. OpenRouter gives real usage through its API. Claude Team accounts: use member usage or cost reporting where the Team admin tools expose it (confirm at https://support.claude.com), else per-seat cost from the Outlook invoice. Outlook invoices read read-only through Graph, reusing the Microsoft 365 app. Keys in the vault, mail read-only, least privilege.

---

13. Integrations, notifications, and access

Access (SSO). Login is company SSO only, through your Microsoft Entra ID (Azure AD) tenant since you are on Microsoft 365, using OIDC or SAML. Restrict to the company email domain so only company accounts can sign in. The SSO identity maps to an employee record by `sso_subject`, which sets role and scope. No external or password signups. Confirm whether you want group-based role mapping from Entra or manual role assignment in Relay.

Outbound channels. A notification layer routes a trigger and target to a channel adapter: in-app, Microsoft Teams now, email later. Nudge on a missed daily report is the first live trigger.

Microsoft 365 (Teams and Outlook, one tenant). Microsoft Graph through Azure AD covers Teams sending and Outlook invoice reading. Map each employee to their Teams id. One caveat: proactive one-to-one Teams messages often need a Teams bot on the Bot Framework. Confirm permissions for your tenant. SSO and Graph are separate concerns but live in the same Entra tenant.

Inbound connectors. OpenRouter API, Anthropic Team usage, and read-only Outlook mail, each an adapter behind a common interface.

---

14. Environments and release (beta and prod)

Two isolated environments.

- **prod.** What users sign in to. Real data, real connectors, real spend.

- **beta.** For building and testing. Separate database, separate secrets, and sandboxed connectors. Beta never messages real employees on Teams, never reads the real billing mailbox, and uses test model keys with a budget cap. Use a test Teams channel and a test mailbox.

Promotion is eval-gated. Prompt, guideline, and workflow-definition changes are tested in beta first. The change reaches prod only after its Engram eval set passes the agreed threshold, so quality cannot silently regress. This is the validate-then-activate step for Codex workflow edits made concrete: validate in beta against evals, then activate in prod.

Use feature flags so a flow can be dark-launched in prod and turned on per department or per role.

---

15. Built for extension

More departments. Department is first-class everywhere. Onboarding product, engineering, or pedagogy is config: add the department, its people and tree, and its guidelines and workflows in the Codex. One shared platform with department isolation, because the ticket flow needs departments to talk to each other.

Ticket and issue flow (later). Cross-department service requests reserved in the `tickets` table: requester, requesting and serving department, type, priority, status, SLA, links to spawned tasks.

Monthly check-in brief (next version). Before your monthly check-in with each L2, the **Briefer** agent compiles a brief from that L2's team: task throughput and on-time rate, blocked items, report and worklog quality, spend, and notable Engram corrections since the last check-in. Stored in `checkin_briefs`. After the check-in, the feedback you give the L2 is captured as tracked items, created as tasks with source = checkin and assigned to the L2, so each piece of feedback is followed to Done. The next brief shows whether the last round of feedback was implemented. This reuses Engram, analytics, and the task system, so it is a thin addition when you ask for it.

---

16. Admin dashboard and L3 views

Admin: guideline proposal inbox with approve or reject and feedback, Codex editing, permission and department and connector and environment config, nudge controls, and the app feedback stream. L3 (you): department task analytics, the blocked tasks board, internal cost and observability dashboards, the combined total AI cost view, eval and agent-quality dashboards, and full Codex edit rights.

---

17. Recommended tech stack

- Backend: Python with FastAPI.
- Database: Postgres as single source of truth, pgvector for search and the memory graph.
- Workflow engine: Temporal at scale, or cron plus a queue to start, driven by `workflow_defs`.
- Agent orchestration: LangGraph with approval interrupts. DSPy for the agents that learn from Engram.
- Observability and eval: OpenTelemetry plus self-hosted Langfuse, with eval sets run on promotion.
- Auth: company SSO through Microsoft Entra ID (OIDC or SAML), domain-restricted.
- Integration: Microsoft Graph and, if needed, Bot Framework. OpenRouter API.
- Secrets: a secret manager or vault.
- LLM: Claude, with cost-based model routing.
- Frontend: React, react-force-graph, and a markdown and diagram renderer for the Codex.
- Environments: separate beta and prod stacks with feature flags.

---

18. Build sequence

- Phase 0: data model, company SSO, reporting tree, permission and department layer, task CRUD, Tasks view, the observability and cost harness, Engram capture wired into the first reviewable action, and the beta and prod split with sandboxed beta connectors.
- Phase 1: Daily report, Nudge, Teams adapter, Agent Farm registry, and the Codex shell.
- Phase 2: MOM flow, Concierge, Cartographer with Second Brain, and the Expense Tracker.
- Phase 3: Weekly Rollup, Monthly Ledger, Sentry, Second Brain query, Anthropic usage connector, first eval sets and eval-gated promotion.
- Phase 4: Curator self-evolving loop reading Engram and writing to the Codex, Admin dashboard, full L3 dashboards, optional Quartermaster.
- Next version: Briefer and the monthly check-in brief.
- Later: second department and the ticket flow.

Engram capture and the environment split are in Phase 0 even though their payoff comes later, because you cannot reconstruct corrections you did not record or safely test in an environment you did not separate.

---

19. Where your knowledge base inputs plug in

Input	Used in
Task Format Guidelines	Dispatcher; Curator; Codex
Daily Report SOP	Daily validation; Rollup; Curator; Codex
Weekly Report Guidelines	Rollup; approval; Codex
Metric Mapping	Categorization; Ledger; analytics; Codex
Team members list (role, id, dept, level, manager, email, Teams id)	employees; tree; scope; SSO mapping; Teams; spend attribution
Unit economics	Estimates; Ledger; Curator checks; capacity
Meeting MOMs	MOM flow; Second Brain
Tool accounts and Outlook invoices	Expense Tracker
Human edits and approvals	Engram, eval sets, Curator proposals

---

20. Open questions

Resolved: Claude on Team plan, billing in Outlook, Codex read-for-all and edit-for-Admin-or-L3, company-SSO-only login, and beta and prod environments.

1. Naming: confirm Relay or pick an alternate.
2. Do team external agents use their own keys, or can they route through a shared gateway for true usage in the Expense Tracker?
3. Entra setup: one app for SSO and a Graph app for Teams and Outlook in your tenant, with a Teams bot if proactive messaging needs it. Can you provision these?
4. SSO roles: map roles from Entra groups, or assign roles manually in Relay?
5. Eval gate: what pass threshold blocks a promotion, and who signs off when a change fails its eval?
6. Engram retention: how long do you keep raw interactions, and is anonymizing the editor in eval sets acceptable?
7. Check-in brief cadence and scope when you build it.